2026S-AML-2203-OTT01 — Advanced Python AI & ML Tools

Final Term Project — Code Review, Quality Assessment & Refactoring
Technical Report

Course	AML-2203 Advanced Python AI & ML Tools
Group	2026S-AML-2203-OTT01-Advanced Python AI & ML - 2
Category	Python-Groups
Codebase	SupportFlow / AML-2404 AI Customer Service Chatbot (Python Flask backend)
Primary modules reviewed	backend/app.py, chatbot.py, database.py, config.py, auth.py, security.py, language_util.py
Static analysis tools	pylint 4.0.6, flake8, black 26.5.1
AI assistant used	Cursor / Copilot-style AI for review suggestions and refactor design

1. Introduction & Project Objective

This report critically evaluates and improves the quality of our semester Python codebase developed for the AML-2404 / SupportFlow customer-service chatbot SaaS. The backend is a multi-module Flask application with JWT auth, SQLite persistence, Stripe billing, RAG-style knowledge retrieval, and local Ollama AI integration. The goal of AML-2203 is not to rebuild the product, but to practice clean-code habits: PEP 8 compliance, modularity, readable control flow, static analysis, and one meaningful refactor grounded in evidence from tools and manual review.

2. Selected Codebase

We selected the SupportFlow Flask backend because it contains substantial logic (auth decorators, multi-tenant database helpers, billing webhooks, chat orchestration, and a large OllamaAIChatbot class). This provides enough surface area for a meaningful review of naming, duplication, complexity, and PEP issues.

- app.py — HTTP routes, CORS/security hooks, billing, chat endpoints
- chatbot.py — intent/RAG/AI response generation and sanitization
- database.py — SQLite schema, sessions, plan-aware public views
- config.py / auth.py / security.py / language_util.py — supporting modules

3. Review Method (Manual + Tools + AI)

3.1 Manual review

We read module boundaries and high-traffic paths (auth, knowledge update, chat prepare/stream, billing sync). We looked for duplicated business rules, unclear names, oversized functions, broad exception handlers, and places where plan limits or timestamps were redefined instead of shared.

3.2 Static analysis tools

We ran flake8 (style / unused names), pylint (complexity, docs, imports), and black --check (formatter drift). Reports are stored under aml-2203-code-review/analysis/.

3.3 AI-assisted feedback

Cursor AI was used to: (1) summarize hotspots in large files, (2) propose candidate refactors with lowest product risk, and (3) draft helper signatures that preserve API behavior. AI suggestions were treated as hypotheses and validated against the live route logic before changing code.

4. Summary of Tool Findings

Tool	Key observations
flake8	17 issues across reviewed files: mostly E501 long lines; also E305 blank-line spacing in config.py, unused global in chatbot cache helper (F824), trailing blank line (W391), and one E203 slice spacing style conflict with black.
pylint (core modules)	Rated about 9.08/10 on auth/config/security/language_util. Common notes: missing docstrings (C0116), import-outside-toplevel (C0415) for lazy imports, broad-exception-caught (W0718), too-many-return-statements in language detect.
pylint (app + database)	Rated about 8.99/10. Hotspots: too-many-branches / statements on billing webhook and sync handlers; many broad Exception catches; logging-fstring-interpolation; missing docstrings on several Flask views.
black --check	Would reformat language_util.py and config.py — formatting is mostly consistent but not fully black-normalized across every module.

5. Written Critique — Key Observations

5.1 Strengths

The backend already shows good modular intent: auth decorators are isolated, security helpers centralize sanitization/rate limits, config owns plan limits, and chat endpoints share _prepare_chat_request. That shared chat preparation is an example of prior successful abstraction.

5.2 Issues found (manual + tools)

1. Duplicated business rules in update_knowledge — the plan word-limit error JSON and business_info defaults were copy-pasted in free-text and legacy structured branches. This risks inconsistent error messages and makes limit changes error-prone (DRY / modularity issue).

2. Oversized route handlers — Stripe confirm/sync/webhook functions exceed pylint branch/statement budgets. Flow is correct but hard to test in isolation.

3. PEP / style drift — long lines (E501), occasional missing blank lines (E305), and black drift in a few modules. Not runtime bugs, but reduce polish.

4. Broad exception handling — many except Exception blocks hide root causes; acceptable at HTTP edges, weaker inside helpers.

5. Duplicate utilities — now_utc() exists in both app.py and database.py; count_words previously imported re inside the function (fixed during cleanup).

6. Documentation debt — pylint C0116 on many small helpers; public helpers would benefit from one-line docstrings for newcomers.

5.3 Suggested improvements (prioritized)

- Extract duplicated knowledge limit / seeding helpers (implemented).
- Split Stripe webhook/sync into pure helpers + thin Flask views.
- Share a single time utility module for UTC timestamps.
- Narrow exception types near Stripe/API boundaries; keep generic catch only at top-level views.
- Run black + flake8 in CI to prevent style regressions.
- Add focused unit tests for plan word limits and auth decorators.

6. Implemented Improvement — One Meaningful Refactor

We chose to refactor knowledge plan-limit enforcement in backend/app.py. This is a real business rule used by paying customers' knowledge bases. Duplication here is more dangerous than a cosmetic rename because inconsistent limits could allow over-quota saves on one code path and reject them on another.

6.1 What changed and why

We introduced two helpers: _knowledge_word_limit_response(...) centralizes counting and the 403 JSON payload; _seed_knowledge_business_info(...) centralizes default business_info fields. Both free-text and legacy structured update paths now call the same helpers. Behavior stays the same for valid/invalid payloads, but future copy or limit changes happen once. We also moved import re to module scope in config.py for count_words (PEP import style).

6.2 Before (duplicated logic)

```
"""
BEFORE refactor — excerpt from backend/app.py (update_knowledge).

Duplicated plan word-limit error payloads and repeated business_info seeding
lived inline in both free-text and legacy structured branches.
"""

@app.route("/businesses/<business_id>/knowledge", methods=["PUT"])
@require_business_owner
def update_knowledge(business_id):
    data = request.get_json(silent=True) or {}
    biz = request.current_business
    effective = config.effective_plan_id(biz.get("plan"), biz.get("subscription_status"))
    limits = config.plan_limits(effective)
    max_words = limits["max_knowledge_words"]

    # Free-text knowledge (preferred)
    if "text" in data or "knowledge_text" in data:
        text = data.get("text")
        if text is None:
            text = data.get("knowledge_text")
        if not isinstance(text, str):
            return jsonify({"error": "text must be a string"}), 400
        text = security.sanitize_knowledge_text(text)
        word_count = config.count_words(text)
        if word_count > max_words:
            return jsonify(
                {
                    "error": (
                        f"Knowledge is {word_count} words; your "
                        f"{effective.title()} plan allows {max_words} words. "
                        "Shorten the text or upgrade on Billing."
                    ),
                    "word_count": word_count,
                    "max_knowledge_words": max_words,
                    "effective_plan": effective,
                }
            ), 403
        warnings = security.knowledge_security_warnings(text)
        knowledge = dict(biz.get("knowledge") or {})
        knowledge["text"] = text
        knowledge.setdefault("business_info", {})
        knowledge["business_info"].setdefault("name", biz["name"])
        knowledge["business_info"].setdefault("website", biz.get("website") or "")
        knowledge["business_info"].setdefault(
            "description", biz.get("description") or ""
        )
        biz = db.update_business(business_id, knowledge=knowledge)
        return jsonify(
            {
                "business": db.public_business_view(biz),
                "warnings": warnings,
                "word_count": word_count,
                "max_knowledge_words": max_words,
```

```
            }
        ), 200

    # Legacy structured object
    knowledge = data.get("knowledge")
    if not isinstance(knowledge, dict):
        return jsonify({"error": "Provide text (string) or knowledge (object)"}), 400
    if isinstance(knowledge.get("text"), str):
        knowledge["text"] = security.sanitize_knowledge_text(knowledge["text"])
        word_count = config.count_words(knowledge["text"])
        if word_count > max_words:
            return jsonify(
                {
                    "error": (
                        f"Knowledge is {word_count} words; your "
                        f"{effective.title()} plan allows {max_words} words. "
                        "Shorten the text or upgrade on Billing."
                    ),
                    "word_count": word_count,
                    "max_knowledge_words": max_words,
                    "effective_plan": effective,
                }
            ), 403
    warnings = security.knowledge_security_warnings(knowledge.get("text") or "")
    info = knowledge.setdefault("business_info", {})
    info.setdefault("name", biz["name"])
    biz = db.update_business(
        business_id,
        knowledge=knowledge,
        name=info.get("name") or biz["name"],
        website=info.get("website") or biz.get("website") or "",
        description=info.get("description") or biz.get("description") or "",
    )
    return jsonify({"business": db.public_business_view(biz), "warnings": warnings}), 200
```

Figure 1. Before — identical word-limit error blocks appear twice in update_knowledge.

6.3 After (shared helpers)

```
"""
AFTER refactor — excerpt from backend/app.py.

Word-limit enforcement and business_info seeding are shared helpers.
"""

def _knowledge_word_limit_response(text: str, effective: str, max_words: int):
    """
    Enforce plan knowledge word limits in one place.

    Returns (word_count, error_response_or_None).
    error_response is a (jsonify(...), status) tuple when over limit.
    """
    word_count = config.count_words(text)
    if word_count <= max_words:
        return word_count, None
    return word_count, (
        jsonify(
            {
                "error": (
                    f"Knowledge is {word_count} words; your "
                    f"{effective.title()} plan allows {max_words} words. "
                    "Shorten the text or upgrade on Billing."
                ),
                "word_count": word_count,
                "max_knowledge_words": max_words,
                "effective_plan": effective,
            }
```

```
        ),
        403,
    )


def _seed_knowledge_business_info(knowledge: dict, biz: dict) -> dict:
    """Ensure knowledge embeds basic business identity fields."""
    info = knowledge.setdefault("business_info", {})
    if not isinstance(info, dict):
        info = {}
        knowledge["business_info"] = info
    info.setdefault("name", biz["name"])
    info.setdefault("website", biz.get("website") or "")
    info.setdefault("description", biz.get("description") or "")
    return knowledge


@app.route("/businesses/<business_id>/knowledge", methods=["PUT"])
@require_business_owner
def update_knowledge(business_id):
    data = request.get_json(silent=True) or {}
    biz = request.current_business
    effective = config.effective_plan_id(biz.get("plan"), biz.get("subscription_status"))
    limits = config.plan_limits(effective)
    max_words = limits["max_knowledge_words"]

    # Free-text knowledge (preferred)
    if "text" in data or "knowledge_text" in data:
        text = data.get("text")
        if text is None:
            text = data.get("knowledge_text")
        if not isinstance(text, str):
            return jsonify({"error": "text must be a string"}), 400
        text = security.sanitize_knowledge_text(text)
        word_count, limit_error = _knowledge_word_limit_response(
            text, effective, max_words
        )
        if limit_error:
            return limit_error
        warnings = security.knowledge_security_warnings(text)
        knowledge = _seed_knowledge_business_info(dict(biz.get("knowledge") or {}), biz)
        knowledge["text"] = text
        biz = db.update_business(business_id, knowledge=knowledge)
        return jsonify(
            {
                "business": db.public_business_view(biz),
                "warnings": warnings,
                "word_count": word_count,
                "max_knowledge_words": max_words,
            }
        ), 200

    # Legacy structured object
    knowledge = data.get("knowledge")
    if not isinstance(knowledge, dict):
        return jsonify({"error": "Provide text (string) or knowledge (object)"}), 400
    word_count = None
    if isinstance(knowledge.get("text"), str):
        knowledge["text"] = security.sanitize_knowledge_text(knowledge["text"])
        word_count, limit_error = _knowledge_word_limit_response(
            knowledge["text"], effective, max_words
        )
        if limit_error:
            return limit_error
    warnings = security.knowledge_security_warnings(knowledge.get("text") or "")
    knowledge = _seed_knowledge_business_info(knowledge, biz)
    info = knowledge.get("business_info") or {}
    biz = db.update_business(
```

```
        business_id,
        knowledge=knowledge,
        name=info.get("name") or biz["name"],
        website=info.get("website") or biz.get("website") or "",
        description=info.get("description") or biz.get("description") or "",
    )
    payload = {"business": db.public_business_view(biz), "warnings": warnings}
    if word_count is not None:
        payload["word_count"] = word_count
        payload["max_knowledge_words"] = max_words
    return jsonify(payload), 200
```

Figure 2. After — helpers own the rule; route focuses on request shape and persistence.

7. AI Tool Feedback Used

AI review suggested several candidates: extract now_utc, split webhook handlers, and remove knowledge-limit duplication. We accepted the knowledge-limit refactor because it scored highest on: (a) clear before/after teaching value, (b) low risk to runtime behavior, and (c) direct improvement to modularity of a business rule. AI-generated helper names were edited for clarity and type of return value (word_count, optional Flask error tuple).

8. Final Reflection — What We Learned

Working on a real semester product made PEP and modularity issues more concrete than toy examples. Static tools quickly highlighted complexity and style debt, but manual review was still required to decide which findings matter for maintainability. Duplication of business rules is more costly than long lines: two identical error payloads can silently diverge. AI tools accelerate discovery and drafting, yet humans must verify that abstractions preserve API contracts. Going forward we would enforce formatter/linter checks earlier and extract helpers as soon as a rule appears a second time (Rule of Three / immediate DRY for critical limits).

9. Folder Contents (Submission Package)

- AML-2203-Code-Review-Technical-Report.pdf — PDF version of this report
- AML-2203-Code-Review-Technical-Report.docx — this Word document
- before/update_knowledge_before.py — original excerpt
- after/update_knowledge_after.py — refactored excerpt
- analysis/ — flake8, pylint, and black outputs
- README.md — how to reproduce the analysis

The live improved code also exists in the main repository at backend/app.py and backend/config.py.

— End of Report — Group 2026S-AML-2203-OTT01 / SupportFlow Capstone Codebase —